

Git and CI/CD: Comprehensive Theory Study Guide

Module 1: Git Basics

- **What is Git?** Git is a free, open-source, distributed version control system (DVCS) designed to handle everything from small to very large projects with speed and efficiency.
 - **Why Git?**
 - **Distributed:** Every developer has a full copy of the project history locally, allowing offline work.
 - **Speed:** Operations are local, making them extremely fast.
 - **Branching model:** Allows multiple parallel streams of work without interference.
 - **Data Integrity:** Uses SHA-1 cryptographic hashes to verify change histories.
 - **Git vs. GitHub**
 - **Git:** The local command-line version control software tool.
 - **GitHub:** A cloud-based hosting service for Git repositories, adding team collaboration tools, issue tracking, and automated CI/CD runners (GitHub Actions).
 - **Git Architecture (The Three States)** Git manages files across three local areas:
 1. **Working Directory:** The actual files you are currently editing on your local disk.
 2. **Staging Area (Index):** A staging file preparing modifications for the next commit snapshot.
 3. **Local Repository (`.git/`):** Stores all committed snapshots as permanent history objects.
 - **Local vs. Remote Repository**
 - *Local Repository:* Stored on your personal workstation.
 - *Remote Repository:* Hosted on a central server (GitHub, GitLab), allowing teams to synchronize changes.
-

Module 2: Basic Git Commands

- `git init`: Initializes a brand new, empty local Git repository in the current folder.
- `git clone <url>`: Copies an existing remote repository from a URL to your local machine.
- `git status`: Shows the state of the working directory and staging area (untracked, modified, or staged files).

- `git add <file>` : Moves changes from the working directory to the staging area.
 - `git commit -m "message"` : Saves staged snapshots permanently to the local repository history.
 - `git push <remote> <branch>` : Uploads local commits to a remote repository (e.g. GitHub).
 - `git pull` : Fetches changes from the remote repository and automatically merges them into the current active branch (equivalent to running `git fetch` followed by `git merge`).
 - `git fetch` : Downloads history updates and references from the remote repository without modifying your local working files.
 - `git checkout <commit_or_branch>` : Switches branches or restores working directory files.
 - `git switch <branch>` : A modern command designed specifically to switch between branches.
 - `git restore <file>` : Discards unstaged modifications in the working directory.
-

Module 3: Branching & Merging

- **What is a Branch?** A branch is a lightweight, moveable pointer to a specific commit. Branches allow developers to isolate feature development without affecting the main code line.
 - **Merging Strategies**
 - **Fast-Forward Merge:** Occurs if the target branch has no new commits since the feature branch was created. Git simply moves the branch pointer forward to the tip of the feature branch.
 - **Three-Way Merge:** Occurs if both branches have diverged with new commits. Git uses a common ancestor commit and creates a new **Merge Commit** to combine the histories.
 - **Merge Conflict & Resolution** Occurs when changes are made to the same lines of the same file on two different branches, and Git cannot automatically determine which version to keep.
 - *Resolution:* Developer opens the flagged file, manually reviews conflict markers (<<<<<<, =====, >>>>>>), updates the code, runs `git add` to stage, and executes `git commit`.
 - **Cherry Pick:** Copying a specific commit from one branch and applying it as a new commit on another branch: `bash git cherry-pick <commit_hash>`
 - **Stash:** Temporarily shelves uncommitted changes (both staged and unstaged) to clean the working directory, allowing you to switch branches without committing unfinished work: `bash git stash # Saves changes git stash pop # Re-applies changes`
 - **Tag:** Creates a permanent reference point (like `v1.0.0`) pointing to a specific commit.
-

Module 4: Advanced Git Operations

- **Rebase (`git rebase <branch>`)** Moves the entire sequence of commits from a feature branch and re-applies them on top of the tip of the target branch. > [!WARNING] > **The Golden Rule of Rebasing:** Never rebase commits that have been pushed to a public/shared remote repository, as it rewrites commit history (alters hashes) and disrupts other developers.
 - **Squash:** Combining multiple small commits into a single, clean commit (usually done during a rebase or pull request merge).
 - **Reflog (`git reflog`):** A log of where the HEAD pointer has been over time. Essential for recovering deleted commits or lost branches.
 - **Bisect:** Uses binary search to find the specific commit that introduced a bug: `bash git bisect start → git bisect bad → git bisect good <last_known_good_commit>`
 - **Blame:** Annotates each line of a file with the author and commit hash of the last modification.
 - **Diff:** Shows line-by-line differences between commits, branches, or working directory files.
 - **Revert vs. Reset**
 - **`git revert <hash>`** : Creates a new commit that applies the exact opposite changes of the target commit. Safest way to undo changes on public remote branches.
 - **`git reset`** : Alters repository state:
 - `--soft` : Moves HEAD pointer, keeps changes in staging area.
 - `--mixed` (Default): Moves HEAD pointer, resets staging area, keeps changes in working directory.
 - `--hard` : Moves HEAD pointer and discards all changes in staging and working directory (destructive).
-

Module 5: GitHub Features

- **Pull Request (PR):** A proposal to merge changes from one branch into another, allowing review, discussion, and automated checks before code integration.
 - **Fork:** Creating a personal copy of another user's remote repository on your own GitHub account.
 - **Protected Branches:** Repository rules preventing direct pushes, requiring reviews and status checks to pass before merging into main branches.
 - **CODEOWNERS:** A file defining specific individuals or teams responsible for reviewing code changes to specific directories.
-

Module 6: GitHub Actions (CI/CD)

A native automation platform built into GitHub to orchestrate CI/CD pipelines.

- **Workflow:** Defined in YAML configurations inside `.github/workflows/`.
 - **Event:** Triggers execution (e.g. `on: push` or `on: pull_request`).
 - **Runner:** The virtual or physical machine executing the workflow jobs (can be GitHub-hosted or self-hosted).
 - **Job:** A set of steps executed sequentially on the same runner. Jobs run in parallel unless dependency orders are defined.
 - **Step:** A single task, which can run a shell command or invoke a pre-built Action.
 - *Example Workflow:*

```
yaml name: Python CI on: [push, pull_request] jobs: test: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - name: Set up Python uses: actions/setup-python@v5 with: python-version: '3.11' - name: Run Tests run: | pip install pytest pytest
```
-

Module 7: Collaboration & Workflows

- **Git Flow:** A strict branching model using `main` (production), `develop` (staging), `feature/*` (development), `release/*` (release preparation), and `hotfix/*` (critical patches).
 - **GitHub Flow:** A lightweight, agile model where developers create feature branches from `main`, open Pull Requests, merge to `main` upon approval, and deploy instantly.
 - **Trunk-Based Development:** Developers merge small, frequent commits directly into a single central branch (`trunk` or `main`), avoiding long-lived feature branches and merge conflicts.
-

Module 8: Security

- **SSH Keys:** Authenticating local git pushes to GitHub without passwords using public/private key pairs.
 - **Signed Commits:** Using GPG/SSH keys to cryptographically sign commits, verifying the identity of the author.
 - **Git Secrets:** Preventing committing sensitive items (API keys, credentials) using pre-commit hooks that scan code before allowing commits.
-

Module 9: Production & Workflows

- **Git LFS (Large File Storage):** Replaces large binary files (videos, models) with text pointers inside Git, storing the actual binaries in external servers to prevent repository bloat.
- **Semantic Versioning (SemVer):** Versioning format: `MAJOR.MINOR.PATCH` (e.g. `v1.2.3`).
- **Git Hooks:** Script hooks executed automatically in response to Git events:
 - *Pre-commit:* Runs checks (linting, tests, secret scanning) before allowing a commit.

- **Conventional Commits:** Structuring commit messages (e.g. `feat: add login page`, `fix: resolve crash`) to automate release log generation.
-

★ High-Yield Interview Questions (Git & GitHub)

1. **What is Git and how does it differ from GitHub?** Git is a local, distributed version control tool. GitHub is a cloud-based hosting platform for hosting Git repositories and managing team collaboration.
2. **Explain Git's three-state architecture.** Git manages files across the: Working Directory (active local files), Staging Area (index preparing the next commit), and Local Repository (permanent commit history).
3. **Merge vs. Rebase?**
 - `git merge` combines branches by creating a new merge commit, preserving chronological history.
 - `git rebase` re-applies feature commits on top of the target branch, creating a clean linear history but rewriting commit hashes. Never rebase public branches.
4. **What is a Merge Conflict and how do you resolve it?** Occurs when conflicting changes are made to the same lines of a file. It is resolved by opening the file, manually editing conflict indicators (`<<<<<<`, `=====`), staging the file (`git add`), and committing.
5. **Git pull vs. Git fetch?** `git fetch` downloads metadata and commits from the remote repository without modifying local working files. `git pull` fetches remote changes and automatically merges them into the current active branch (`fetch` + `merge`).
6. **What does git stash do?** Temporarily saves (shelves) uncommitted working changes, reverting the directory to clean HEAD status so you can switch branches without committing incomplete work. Use `git stash pop` to restore.
7. **What is a Cherry-Pick operation?** Applying a specific commit from one branch onto another branch as a new commit: `git cherry-pick <commit_hash>`.
8. **Git revert vs. git reset --hard?** `git revert` creates a new commit that undoes the changes of a target commit. It is safe for public branches. `git reset --hard` deletes commits and discards changes destructively, altering history.
9. **What are Git Hooks and give an example.** Scripts executed automatically on Git events. A *pre-commit* hook can run code formatting, linters, or secret scanners before allowing a commit.
10. **Explain GitHub Actions workflow components.** Defined in YAML. Triggered by **Events** (push/PR). Workflows contain **Jobs** that run on virtual **Runners**, executing sequential **Steps** (shell commands or pre-built Actions).



Module Summary & Key Takeaways

Theory Focus: Version Control Architectures, Advanced Git Workflows, and GitHub Actions (CI/CD).

This module reviews production version control workflows and deployment automation. Core principles include:

- **Git Architecture & Lifecycle:** Code commits transition through the local workspace, staging area (index), local database, and remote repository.
- **Advanced Operations:** Branch maintenance is performed using merges or rebasing (re-applying local changes on top of target tips to preserve linear histories). ``git cherry-pick`` ports specific commits, and ``git reflog`` acts as a recovery log of head movements to restore deleted branches.
- **CI/CD Pipelines:** Continuous Integration automates code validation (linting and testing), and Continuous Deployment automates pipeline builds to target clouds. **GitHub Actions** defines these execution steps via YAML workflows triggered by repository actions.